Effect of Collective Communication Algorithms on LLM training

Anand MT25014
Videh Sharma MT25052
Arjun Prasad MT25064
Yash Verma MT25092

INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI

Problem

- LLM training scales to billions/trillions of parameters - requires multi-GPU training.
- Frequent inter-GPU communication (eg All Reduce) becomes major bottleneck.
- Performance depends on :
  - Communication algorithm (Ring vs NVLS - Tree))
  - System parameters (GPU count, message size, bandwidth)
- Real world evaluation is expensive and time-consuming

Motivation

- Large scale training (eg GPT-3) : ~34 days on 1000+ GPUs
- Small communication improvements lead to huge reduction in training time
- Simulators (SimAI)
  - Replicate distributed training behaviour
  - Provides near real accuracy (~98%)

Objectives

- Compare Collective Communcation strategies
  - Ring-AllReduce vs NVLS-(tree based) AllReduce
  - Across GPU count, message size, bandwidth, sequence length etc
- Analyze parallelism impact
  - Data parallelism
  - Tensor parallelism
  - Pipeline parallelism

Methodology

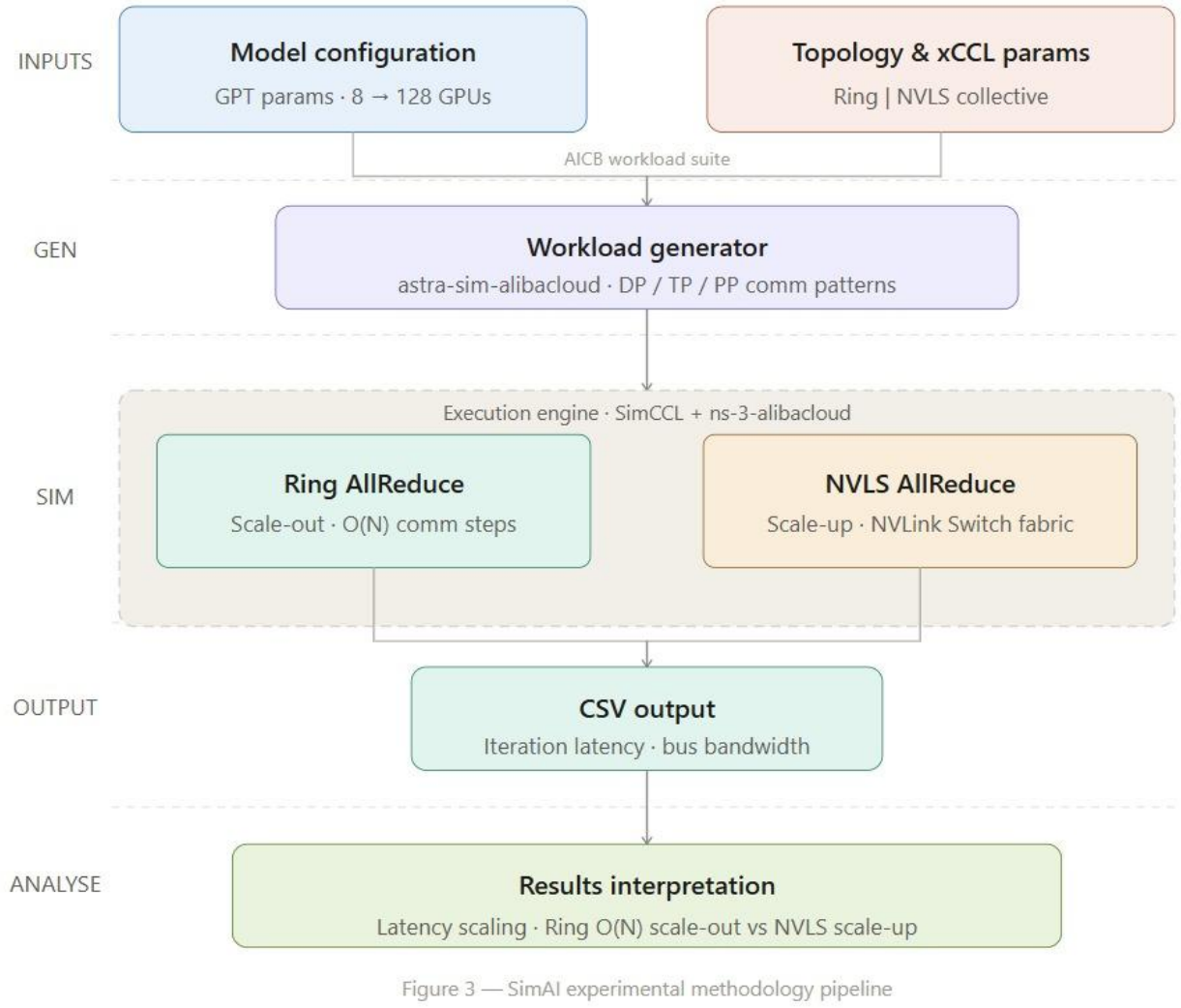


Figure 3 — SimAI experimental methodology pipeline

Implementation

Stage 1 — Hypothesis Formation

- Claim must have three parts: predicted direction, scope condition, measurable outcome
- State the mechanism in one sentence before running anything — prevents post-hoc fitting
- Document formally: algorithm, collective type, message size range, expected BusBW gap

Stage 2 — Workload & Topology Generation

- Two artifacts per experiment: workload `.txt` and topology file
- Workload controls: collective type, message sizes, compute time, parallelism group
- Topology controls: GPU count, NVLink BW, NIC BW, switch configuration

Stage 3 — Simulation Execution

- Binary: `SimAI_simulator` — three key knobs: `AS_NVLS_ENABLE`, `AS_PXN_ENABLE`, `AS_SEND_LAT`
- Two output CSVs per run: `ncclFlowModel_EndToEnd.csv` and `ncclFlowModel_detailed_*.csv`
- Save logs with `tee` for traceability

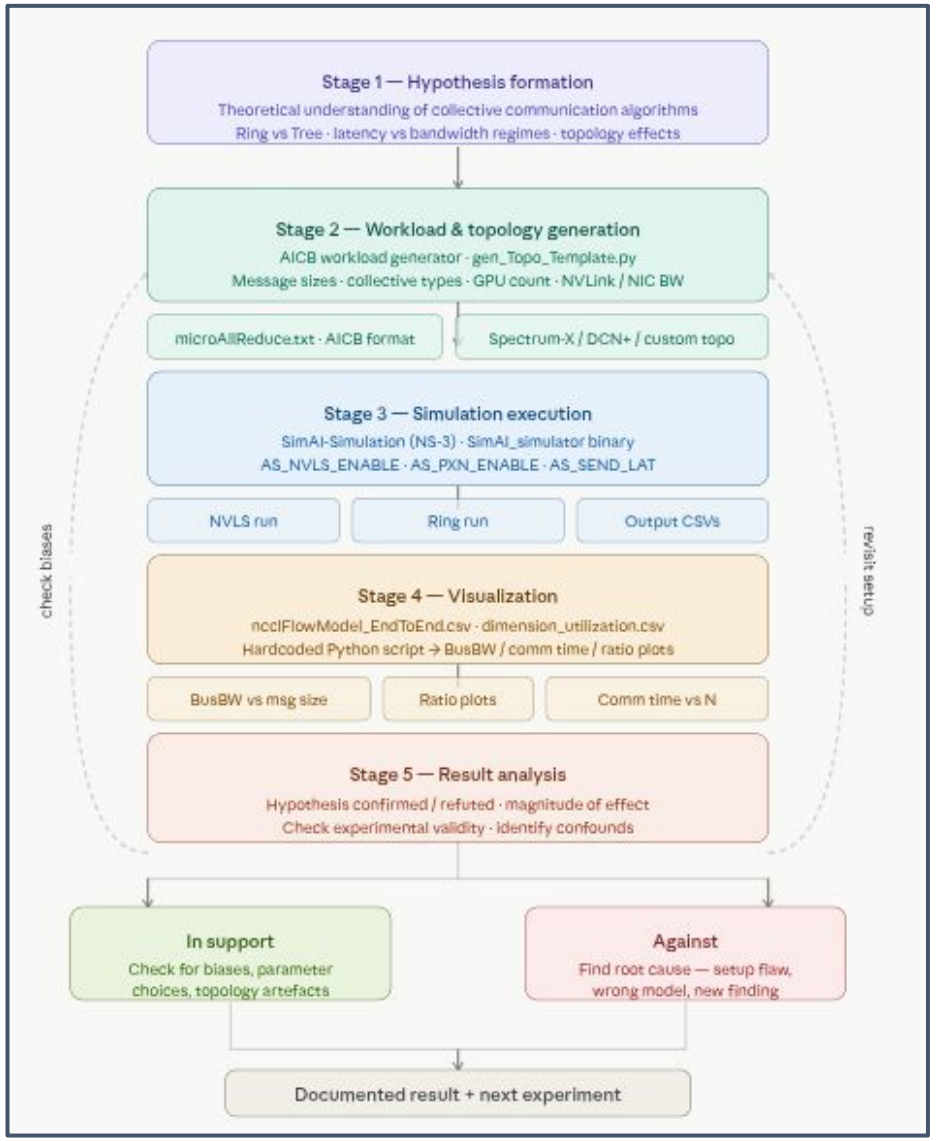
Stage 4 — Visualization

- Extract CSV values and hardcode into Python script — permanent, reproducible record
- Plots: BusBW vs message size, absolute comm time, NVLS/Ring ratio, AR/AG/RS ratio check

Stage 5 — Result Analysis

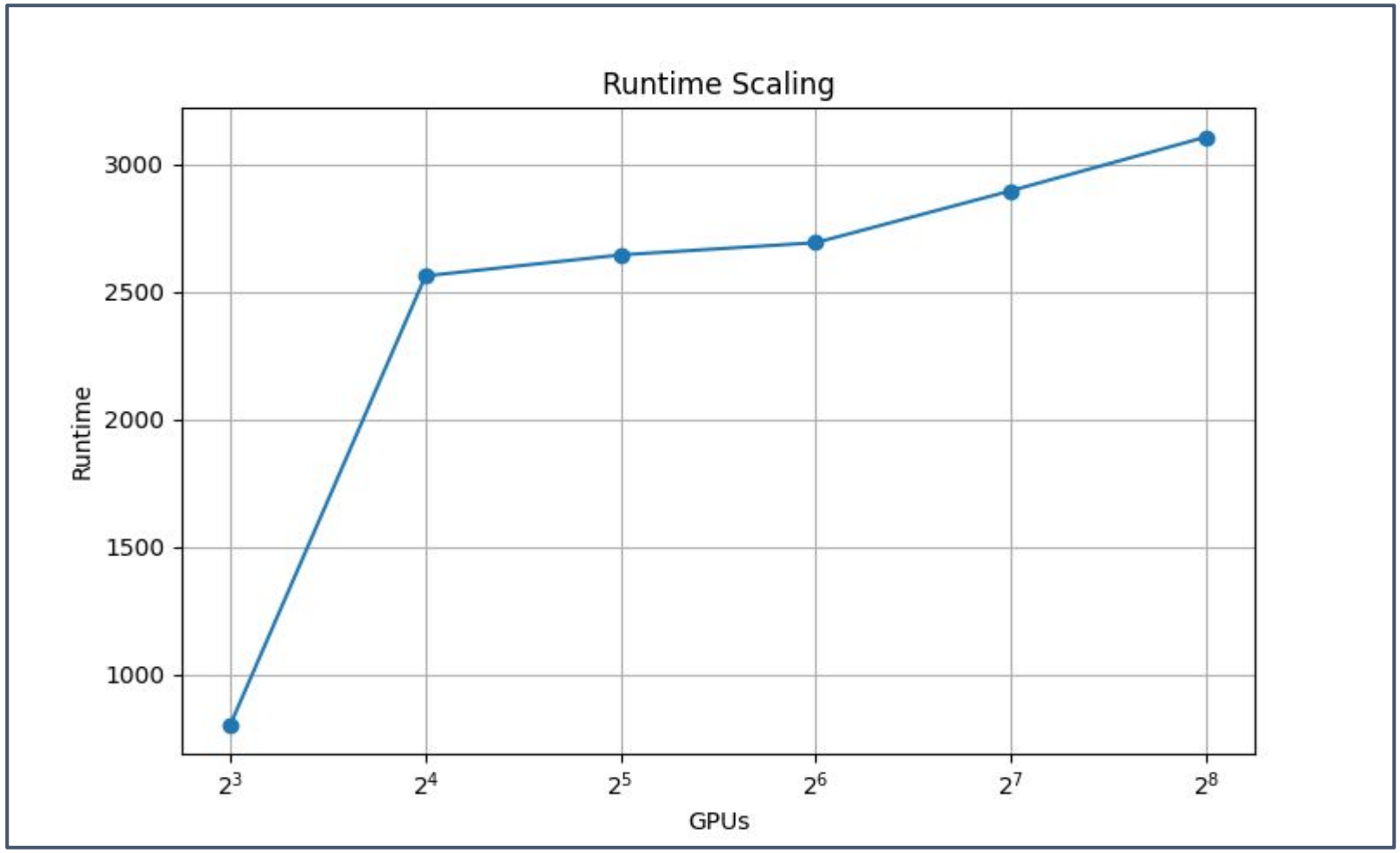
- If result supports hypothesis:
  - Check `AS_SEND_LAT` symmetry between algorithms
  - Check `-nvbw` is not tuned favorably for the winner
  - Verify result holds across all message sizes and at a different GPU count
- If result contradicts hypothesis:
  - First eliminate setup flaws — topology mismatch, bandwidth asymmetry, SimAI modeling limits
  - If setup is clean, the contradiction is the finding — reframe as the next hypothesis

Implementation



Observations

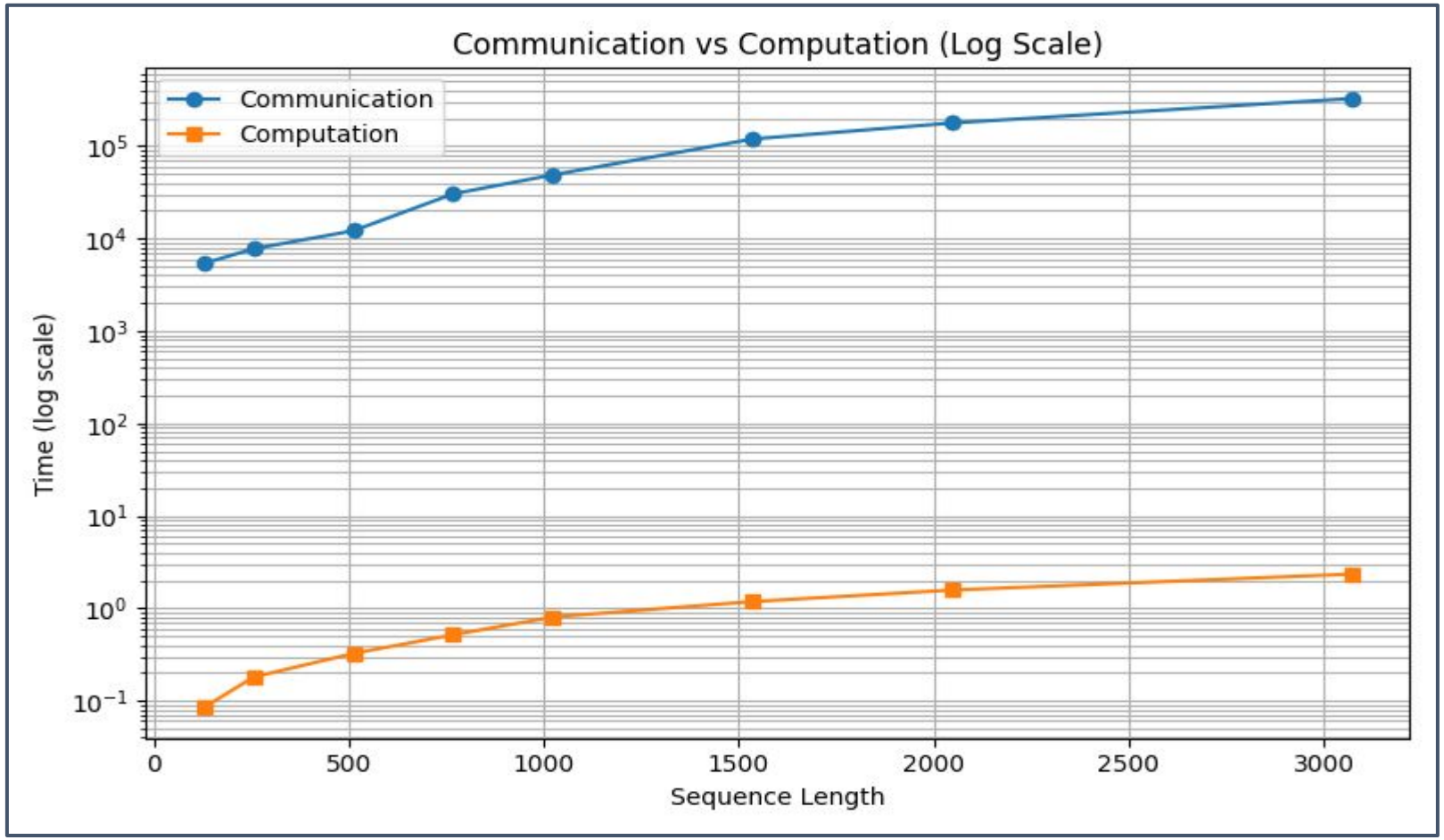
Execution time should reduce with increase in GPUs, observations are otherwise.



workload_constant_hybrid_ring_default_gpus_2_4_8_16_32_64_128_512

Observations

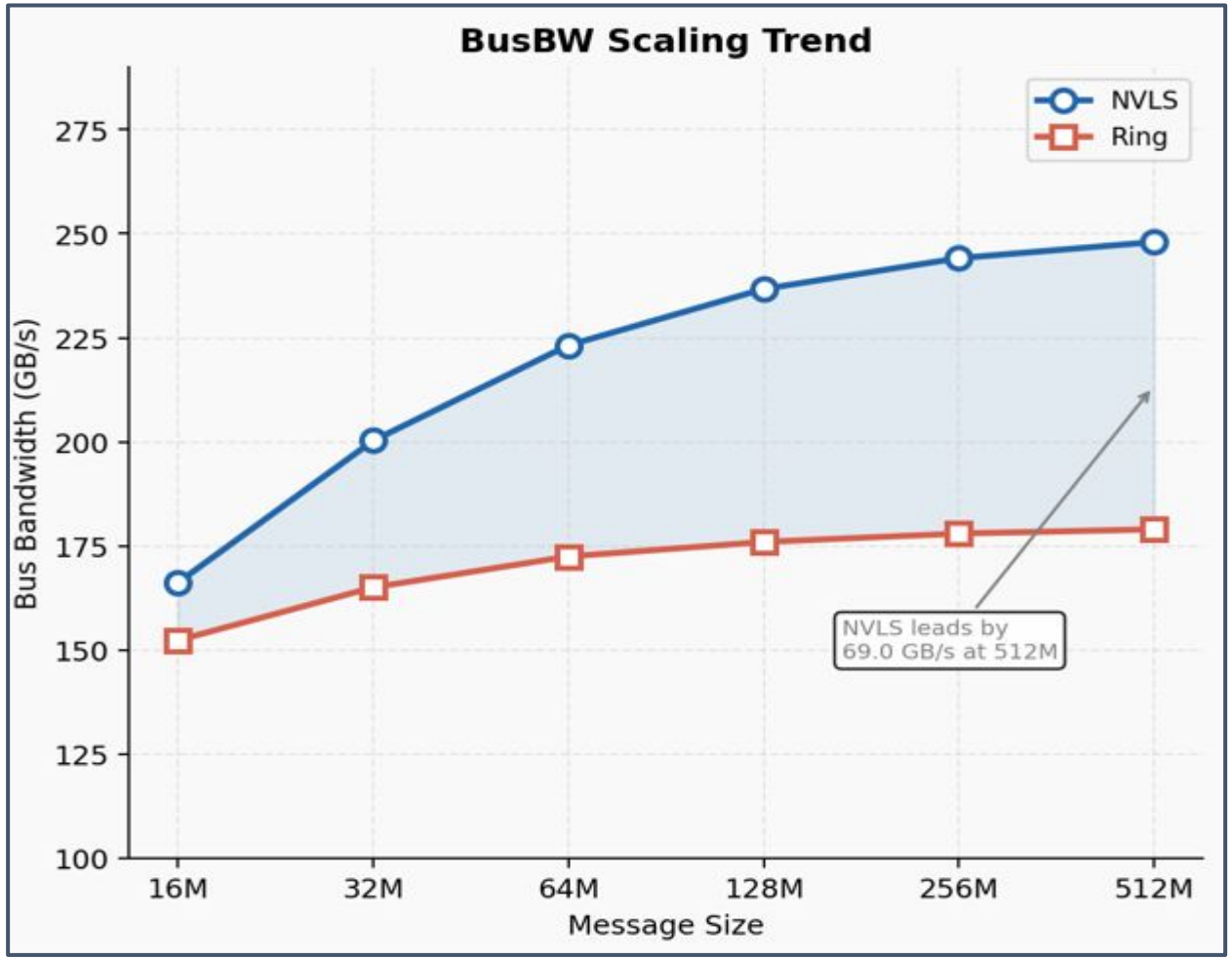
At small sequence lengths communication time dominates, then computation time increases.



gpu cnt 8 sequence length 128 256 512 768 1024 1536 2048 3072

Observations

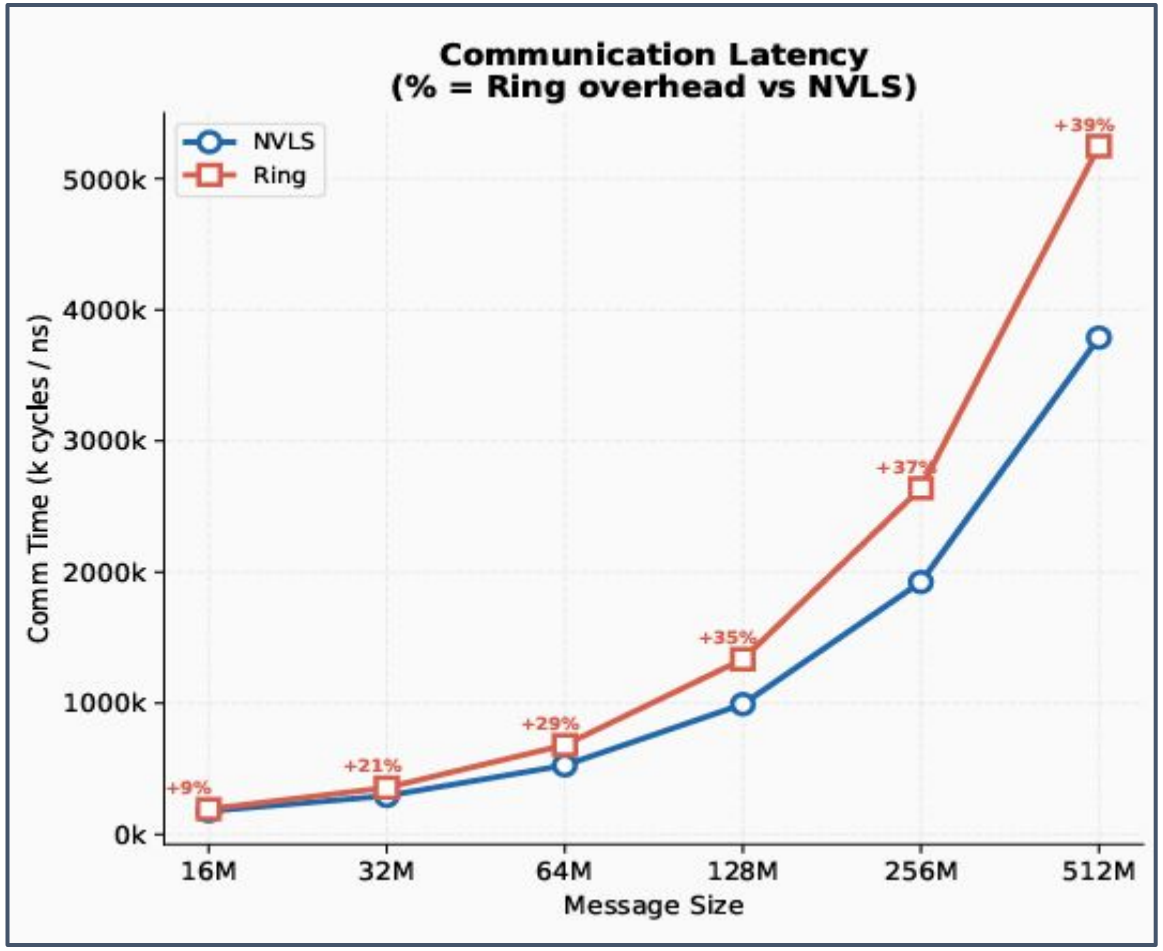
NVLS dominates bus bandwidth across all message sizes



ring-AllReduce_NVLS_AllReduce_32_gpus_H100_rail_optimized_single_tor

Observations

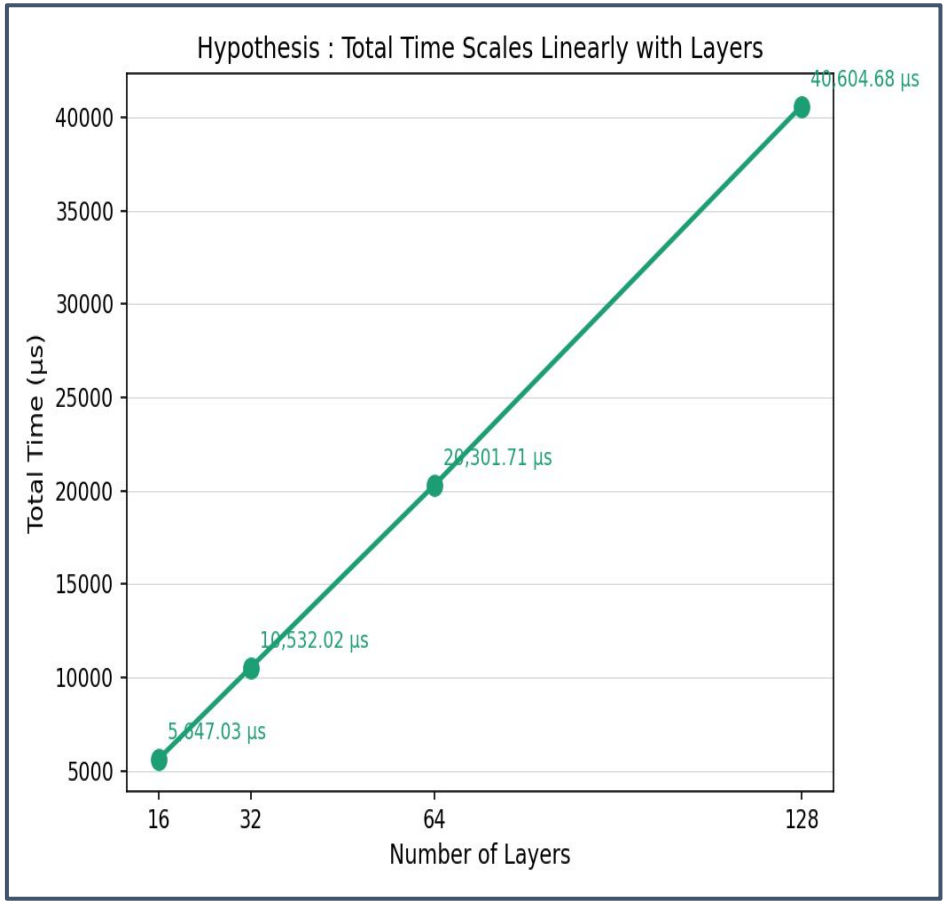
NVLS latency advantage over ring across increasing message sizes

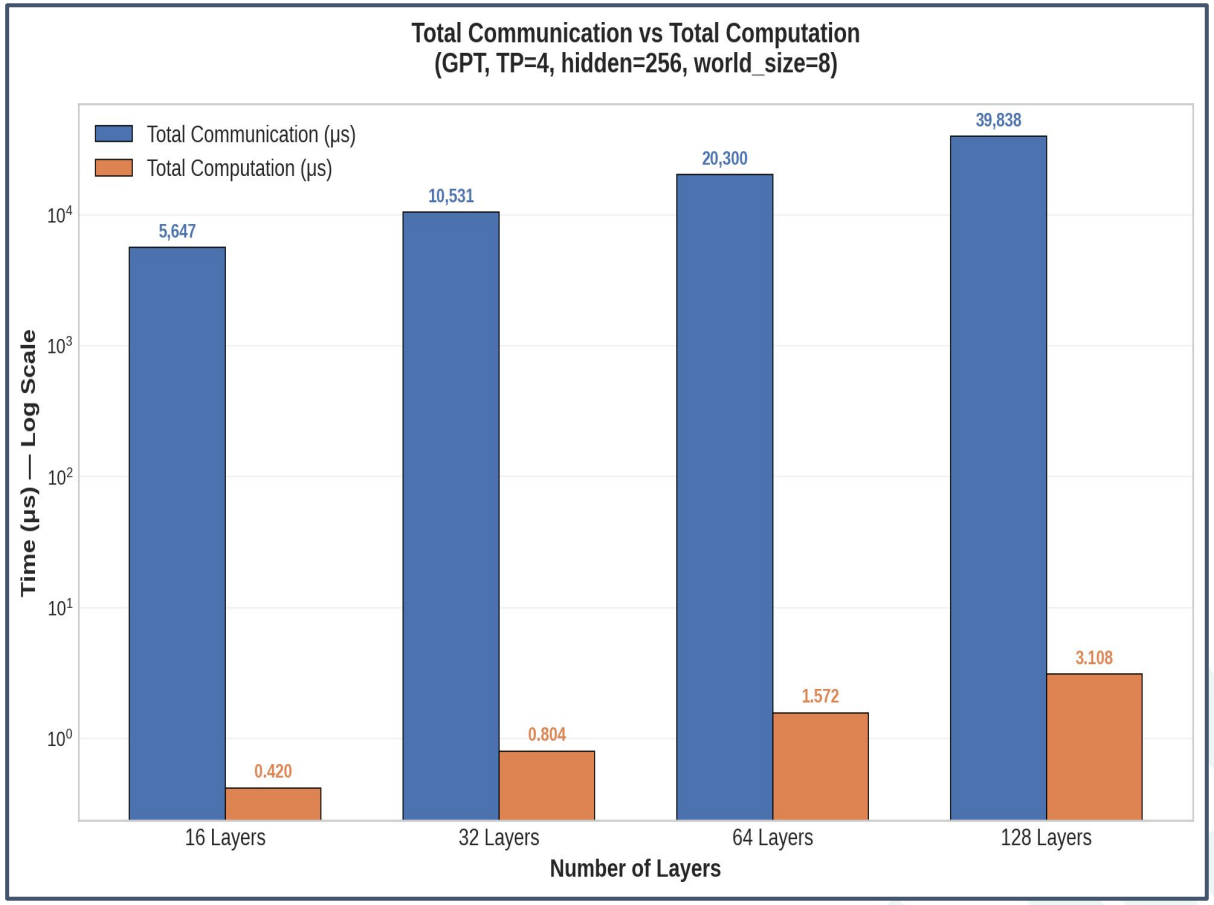


ring-AllReduce_NVLS_AllReduce_32_gpus_H100_rail_optimized_single_tor

Observations

Total execution time scales linearly with number of hidden layers

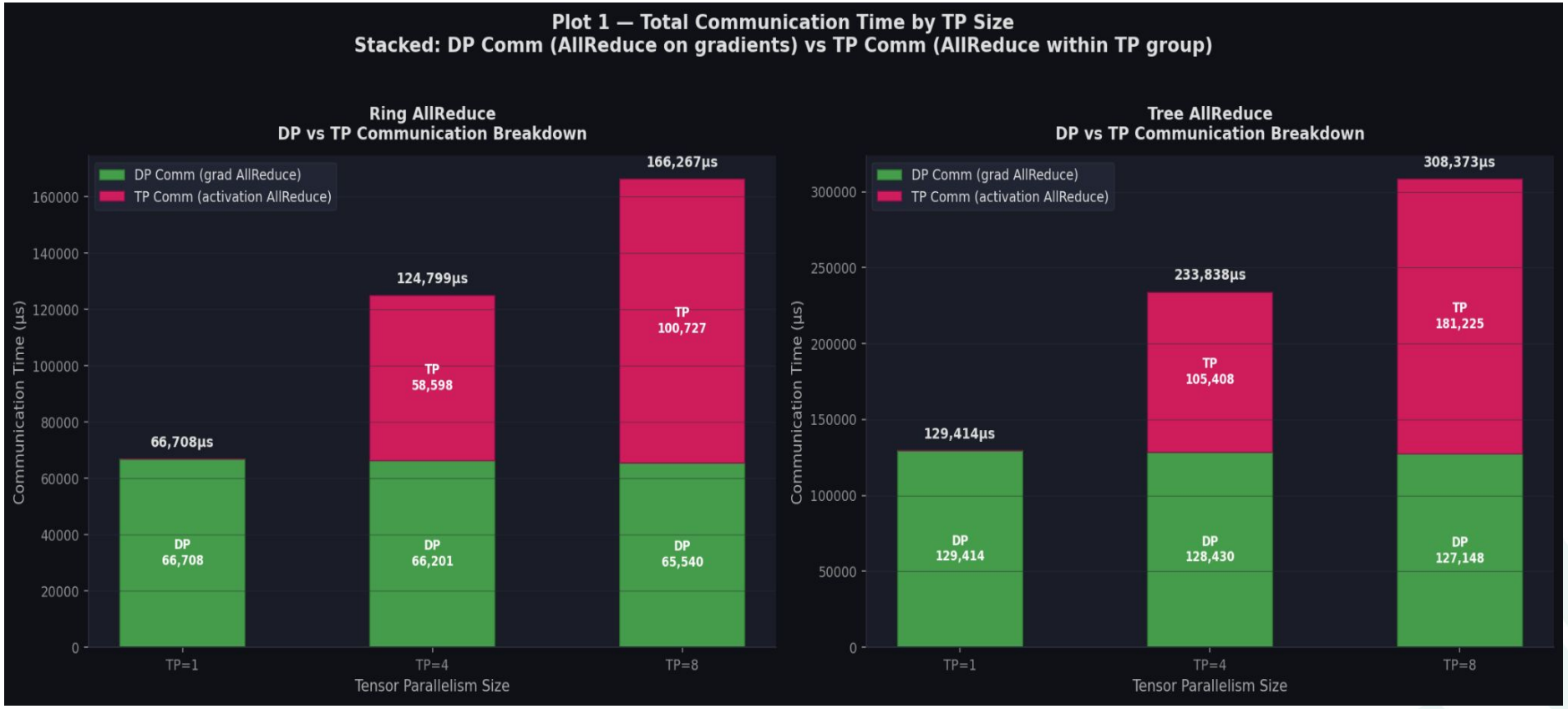




NVLS_Spectrum_32_num_layers_16_32_64_128

Observations

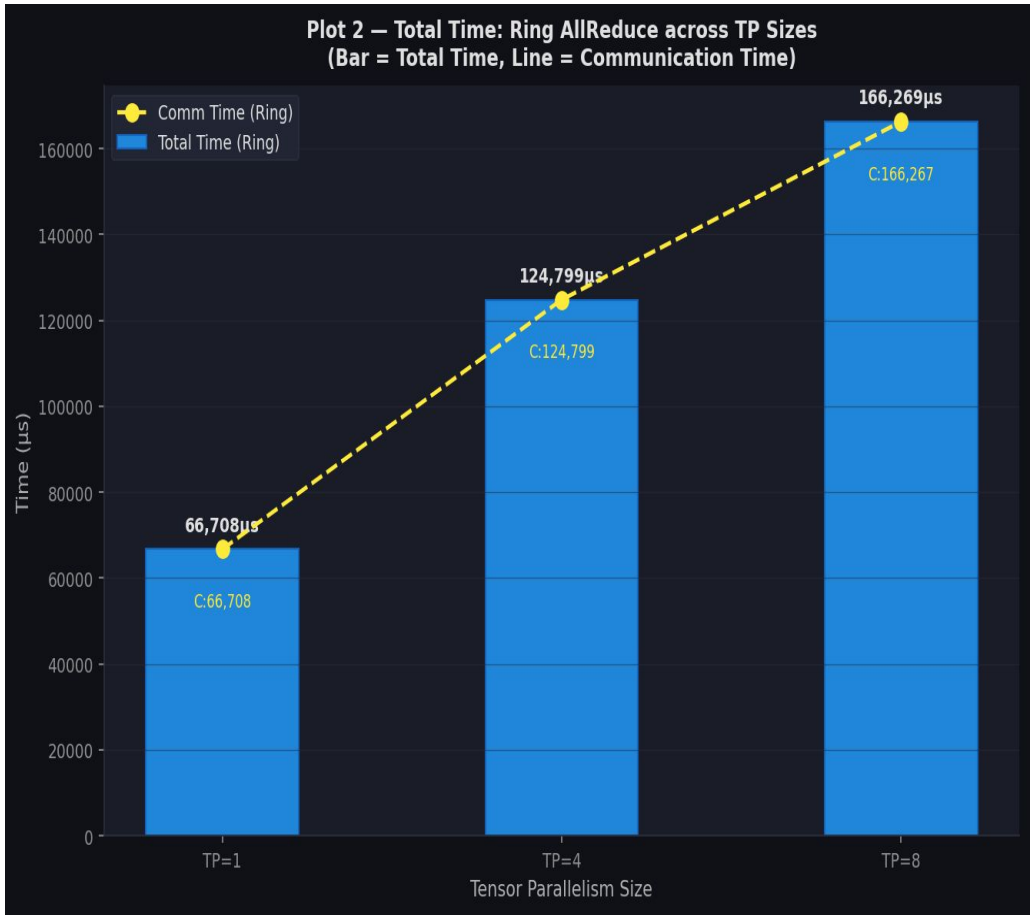
Higher TP increases frequency and size of AllReduce → magnifies differences between algorithms.

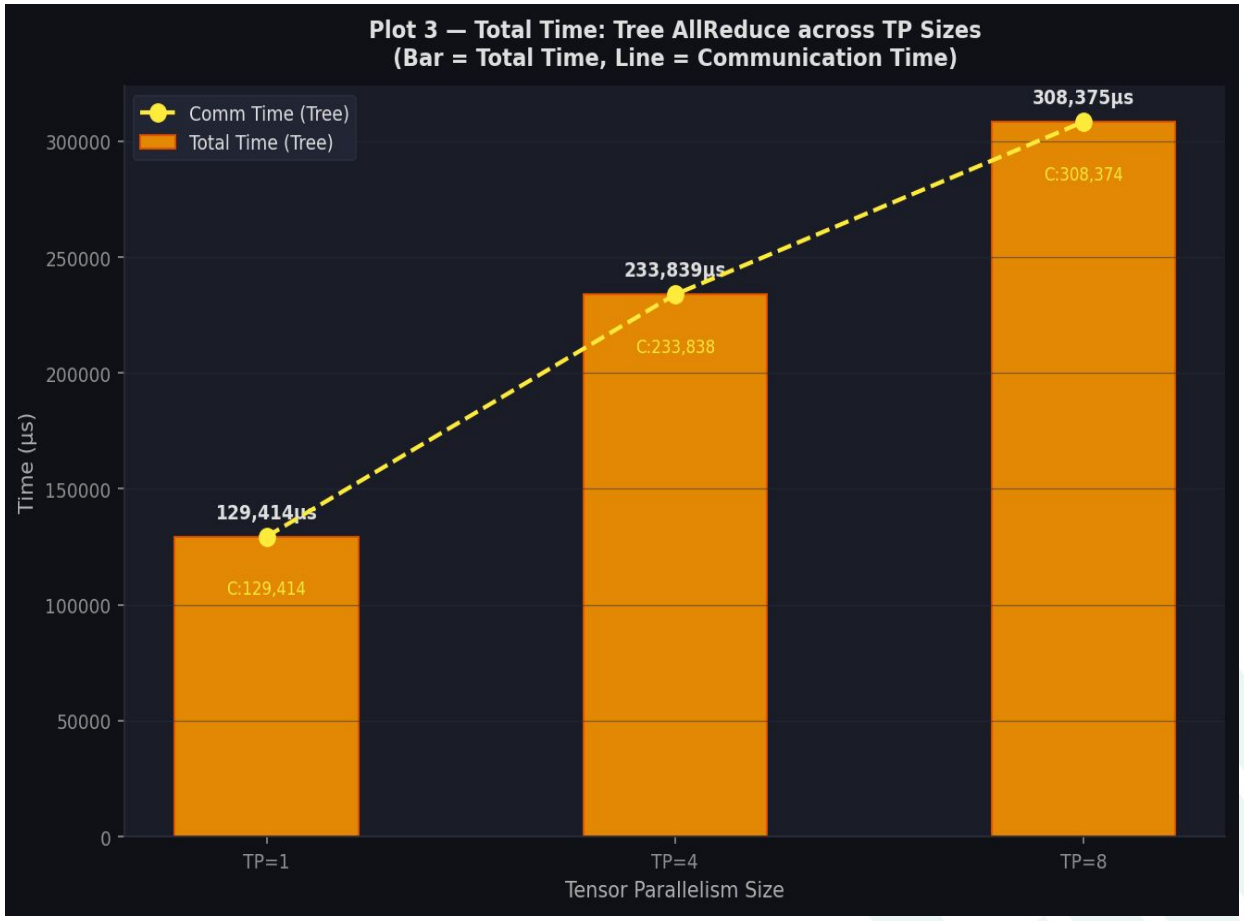


Spectrum-X_gpus_256_TP_1_4_8

Observations

Higher TP increases frequency and size of AllReduce → magnifies differences between algorithms.

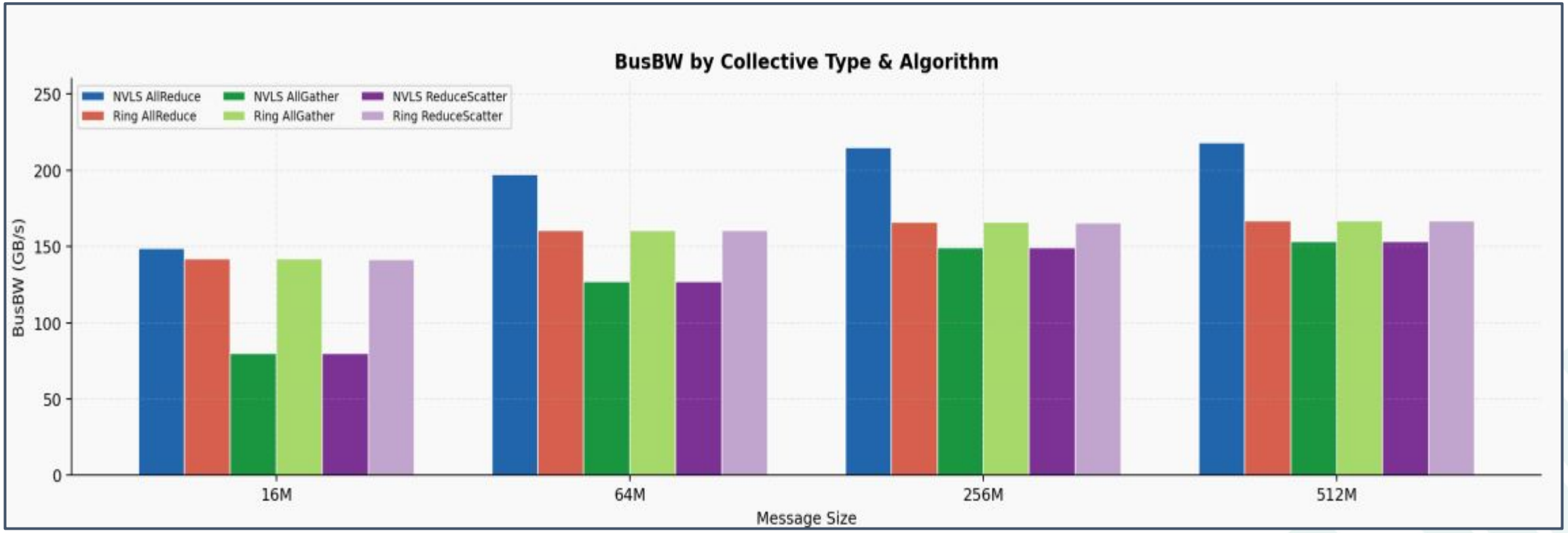




Spectrum-X_gpus_256_TP_1_4_8_ring_vs_nvls

Observations

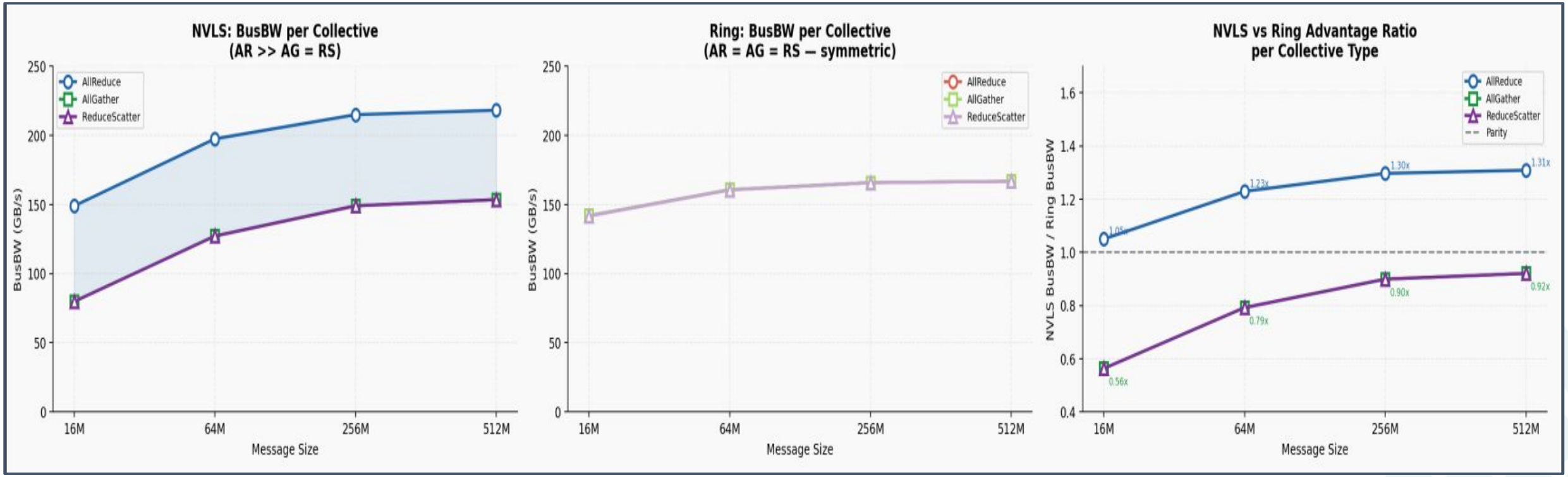
NVLS's hardware tree acceleration is collective-specific — it wins AllReduce by 31% but loses AllGather and ReduceScatter to Ring, meaning the net benefit of NVLS in real LLM training (which uses all three) is significantly smaller than AllReduce



Nvls_ring_message_size_16_64_256_512_M

Observations

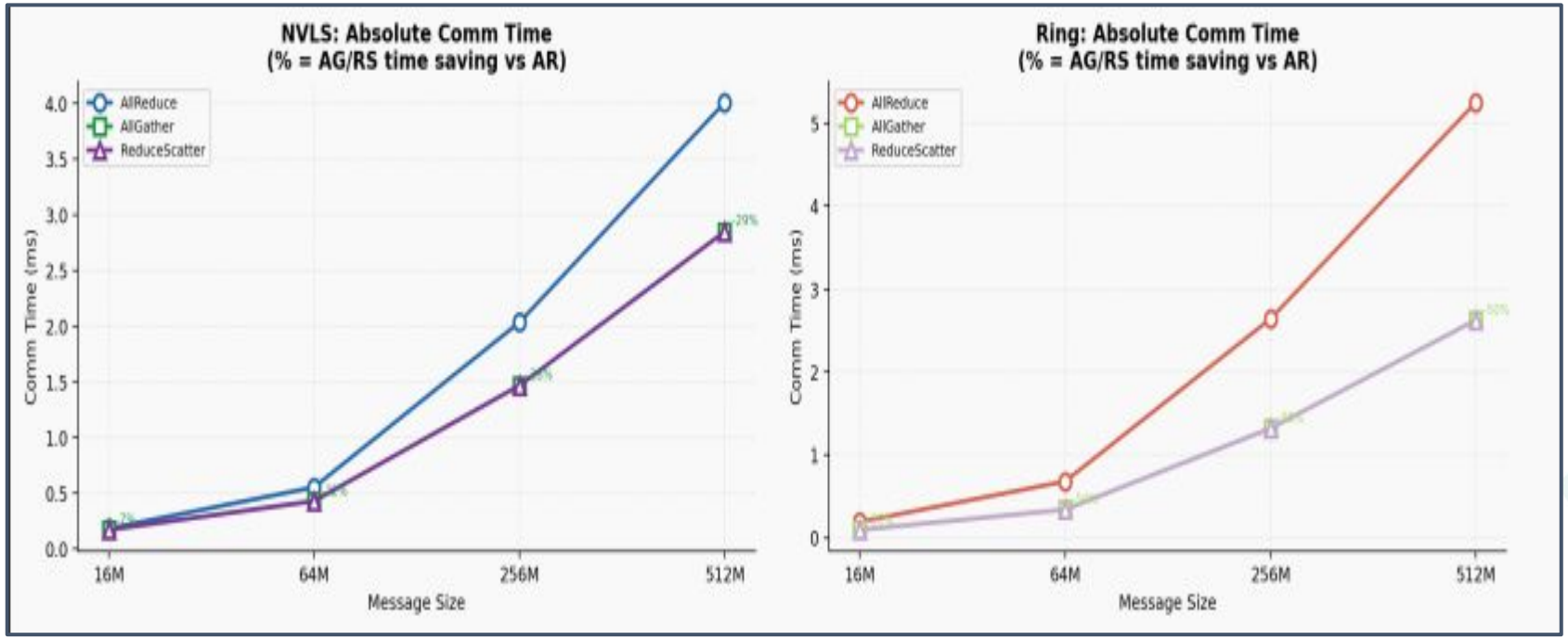
NVLS's hardware tree acceleration is collective-specific — it wins AllReduce by 31% but loses AllGather and ReduceScatter to Ring, meaning the net benefit of NVLS in real LLM training (which uses all three) is significantly smaller than AllReduce



Nvls_ring_message_size_16_64_256_512_M

Observations

NVLS's hardware tree acceleration is collective-specific — it wins AllReduce by 31% but loses AllGather and ReduceScatter to Ring, meaning the net benefit of NVLS in real LLM training (which uses all three) is significantly smaller than AllReduce



Nvls_ring_message_size_16_64_256_512_M

Conclusion

- NVLS provides significant acceleration for AllReduce, but its performance is collective-specific and does not consistently outperform Ring for AllGather and ReduceScatter.
- The overall benefit of NVLS in realistic LLM training is smaller than its AllReduce advantage, since multiple collectives are involved.
- Increasing TP increases the frequency and size of AllReduce, magnifying differences between algorithms.
- Communication strategy becomes more critical at higher TP configurations.
- Total execution time scales linearly with the number of layers when all other factors remain constant.
- With increasing message size, NVLS shows higher latency than Ring but achieves better bandwidth scaling.
- This reflects a trade-off between latency and throughput.
- At small sequence lengths, communication time dominates, making algorithm choice more important.
- As sequence length increases, computation dominates and communication impact reduces.
- No single communication strategy is universally optimal.
- Performance depends on parallelism configuration, message size, and sequence length.

Thank You!